

Training and Evaluation

This method is really useful when it comes to matrix multiplication with one-hot vectors because it saves us from doing a bunch of unnecessary computation that will return 0 anyway. An illustration from [Chris McCormick](#) for multiplication of a one-hot vector with a matrix.

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 \end{bmatrix} \times \begin{bmatrix} 17 & 24 & 1 \\ 23 & 5 & 7 \\ 4 & 6 & 13 \\ 10 & 12 & 19 \\ 11 & 18 & 25 \end{bmatrix} = \begin{bmatrix} 10 & 12 & 19 \end{bmatrix}$$

So, to get the embedding (or vector representation) of the input center words, we use this:

```
embed = tf.nn.embedding_lookup(embed_matrix, center_words, name='embed')
```

4. Define the loss function

While NCE is cumbersome to implement in pure Python, TensorFlow already implemented it for us. It is worth knowing why the library version exists at all. Noise contrastive estimation replaces the full softmax denominator with a sampled approximation, which turns a sum over the entire vocabulary into a sum over a handful of noise words. Written naively in Python that sampling dominates the training loop and undoes the saving entirely, which is exactly why it lives in the framework rather than in your model code.

Note that by the way the function is implemented, the third argument is actually inputs, and the fourth is labels. This ambiguity can be quite troubling sometimes, but keep in mind that TensorFlow is still new and growing and therefore might not be perfect. Nce_loss source code can be found [here](#).

For nce_loss, we need weights and biases for the hidden layer to calculate NCE loss. They will be updated by optimizer during training. After sampling, the final output score will be computed as followed. This computation is done internally in tf.nn.nce_loss operation.

Then we define loss:

5. Define optimizer

We will use the good old gradient descent.

Phase 2: Execute the computation

We will create a good old session to run the optimizer to minimize the loss, and report the loss value back to us. Don't forget to reinitialize your iterator!

You can see the full basic model on the class's GitHub repo under the name `word2vec.py`.

Interface: How to structure your TensorFlow model

All models we've built so far have more or less have the same structure:

Phase 1: assemble your graph

1. Import data (either with `tf.data` or with placeholders)
2. Define the weights
3. Define the inference model
4. Define loss function
5. Define optimizer

Phase 2: execute the computation

Which is basically training your model. There are several steps to it:

1. Initialize all model variables for the first time.
2. Initialize iterator / feed in the training data.
3. Execute the inference model on the training data, so it calculates for each training input example the output with the current model parameters.
4. Compute the cost
5. Adjust the model parameters to minimize/maximize the cost depending on the model.

Here is a visualization of training loop from the book *TensorFlow for Machine Intelligence* (Abrahams et al., 2016).

It took us about 20 lines of code to build a word embedding model! It's fast but ... "what happened to decomposition?" After we've spent an ungodly amount of time building a model, we'd like to use it more than once.

Question: how do we make our model most easy to reuse?

Hint: take advantage of Python's object-oriented-ness.

Answer: build our model as a class!

Our model base class should follow the interface. We combined step 3 and 4 because we want to put embed under the name scope of "NCE loss".

Keeping the phases in the same order as the interface is what makes a second model cheap to write: the skeleton is already there and only the inference step changes.

Visualize embeddings

Now let's see what our model finds after training it for 100,000 iterations. If we visualize our embedding with t-SNE, we will see something like this.

It's hard to see in 2D, but we'll see in class in 3D that all the number (one, two, ..., zero) are grouped in a line on the bottom right, next to all the alphabet (a, b, ..., z) and names (john, james, david, and such). All the months are grouped together. "Do", "does", "did" are also grouped together and so on.

If you print out the closest words to 'american', you will find its closest cosine neighbors are 'british' and 'english'. Fair enough.

How about words closest to ‘government’?

If you haven’t used t-SNE, you should start using it! It’s super cool.

You can visualize more than word embeddings, aka, you can visualize any vector representations of anything! Have you read Chris Olah’s blog post about [visualizing MNIST](#)? t-SNE made MNIST desirable! Image below is from Olah’s blog.

We can also visualize our embeddings using PCA too.

And we did all that visualization with less than 30 lines of code using TensorFlow projector with TensorBoard! The tool is super useful, albeit a bit finicky to use. The visualization will be stored in visualization folder. To see it, run `''tensorboard --logdir='visualization''`.

To see the full code, please see [examples/04_word2vec_visualize.py](#) on the class’s GitHub repo.

Variable sharing

Name scope

Let’s give the tensors name and see how our word2vec model looks like on TensorBoard.

As you can see in the graph, the nodes are scattering all over, rendering the graph difficult to read. TensorBoard doesn't know which nodes are similar to which nodes and should be grouped together. This can make debugging your graph daunting when you build complex models with hundreds of ops.

How can we let TensorBoard know which nodes should be grouped together? For example, we would like to group all ops related to input/output together, and group all ops related to NCE loss together. TensorFlow lets us do that with `name_scope`. You can put all the ops that you want to group together under the block:

For example, our graph can have four name scopes: “data”, “embed”, “loss”, and “optimizer”.

When you visualize that on TensorBoard, you will see your nodes are grouped into neat blocks.

You can click on the plus sign on top of each name scope block to see all the ops inside that block. Take your time to play around with it.

To see the full model of word2vec with name scope defined, see [examples/04 word2vec visualize.py](#) on the class's GitHub repo.

Variable scope

One of the questions I'm often asked is: “So what's the difference between `name_scope` and `variable_scope`”. While both create namespaces, the main thing `variable_scope` does is to facilitate variable sharing. Let's explore why we need variable sharing.

Assume that we want to create a neural network with two hidden layers as followed. We then called that two hidden layers network on two different input `x1` and `x2`.

If we visualize this on TensorBoard, this is what we see.